lasso_hipertensao

April 1, 2026

[ ]:
```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegressionCV

from sklearn.metrics import (
    accuracy_score,
    roc_auc_score,
    classification_report,
    confusion_matrix,
    RocCurveDisplay
)

import matplotlib.pyplot as plt
import seaborn as sns

from imblearn.over_sampling import SMOTE

from sklearn.metrics import precision_recall_curve, f1_score
```

[92]:
```
df = pd.read_csv('../data/processed/pns_2019_final.csv')
df.shape
```

[92]: (87657, 78)

[93]:
```
df = df.dropna()
df.shape
```

[93]: (74304, 78)

[94]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
Index: 74304 entries, 0 to 87655
Data columns (total 78 columns):
```

```
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   situacao_domicilio      74304 non-null  int64
 1   acesso_internet         74304 non-null  int64
 2   plano_saude             74304 non-null  float64
 3   hipertensao             74304 non-null  float64
 4   diabetes                74304 non-null  float64
 5   doenca_coracao          74304 non-null  float64
 6   infarto                 74304 non-null  float64
 7   angina                  74304 non-null  float64
 8   insuficiencia_cardiaca  74304 non-null  float64
 9   arritmia                74304 non-null  float64
 10  avc                     74304 non-null  float64
 11  asma                    74304 non-null  float64
 12  artrite                 74304 non-null  float64
 13  dort                    74304 non-null  float64
 14  depressao               74304 non-null  float64
 15  doenca_pulmao           74304 non-null  float64
 16  enfisema                74304 non-null  float64
 17  bronquite               74304 non-null  float64
 18  insuficiencia_renal     74304 non-null  float64
 19  infra_agua              74304 non-null  int64
 20  sexo_feminino           74304 non-null  int64
 21  n_doencas               74304 non-null  int64
 22  doenca_cronica          74304 non-null  int64
 23  uf_AL                   74304 non-null  bool
 24  uf_AM                   74304 non-null  bool
 25  uf_AP                   74304 non-null  bool
 26  uf_BA                   74304 non-null  bool
 27  uf_CE                   74304 non-null  bool
 28  uf_DF                   74304 non-null  bool
 29  uf_ES                   74304 non-null  bool
 30  uf_GO                   74304 non-null  bool
 31  uf_MA                   74304 non-null  bool
 32  uf_MG                   74304 non-null  bool
 33  uf_MS                   74304 non-null  bool
 34  uf_MT                   74304 non-null  bool
 35  uf_PA                   74304 non-null  bool
 36  uf_PB                   74304 non-null  bool
 37  uf_PE                   74304 non-null  bool
 38  uf_PI                   74304 non-null  bool
 39  uf_PR                   74304 non-null  bool
 40  uf_RJ                   74304 non-null  bool
 41  uf_RN                   74304 non-null  bool
 42  uf_RO                   74304 non-null  bool
 43  uf_RR                   74304 non-null  bool
 44  uf_RS                   74304 non-null  bool
 45  uf_SC                   74304 non-null  bool
```

```
 46  uf_SE                   74304 non-null  bool
 47  uf_SP                   74304 non-null  bool
 48  uf_TO                   74304 non-null  bool
 49  domicilio_Apartamento   74304 non-null  bool
 50  domicilio_Cortico       74304 non-null  bool
 51  domicilio_Ignorado      74304 non-null  bool
 52  raca_Preta              74304 non-null  bool
 53  raca_Amarela            74304 non-null  bool
 54  raca_Parda              74304 non-null  bool
 55  raca_Indigena           74304 non-null  bool
 56  raca_Ignorado           74304 non-null  bool
 57  escolaridade_ord        74304 non-null  float64
 58  imc                     74304 non-null  float64
 59  tabagismo_status        74304 non-null  int64
 60  tabagismo_ativo         74304 non-null  bool
 61  alcool_status           74304 non-null  int64
 62  alcool_ativo            74304 non-null  bool
 63  atividade_status        74304 non-null  int64
 64  atividade_ativo         74304 non-null  bool
 65  faixa_etaria            74304 non-null  str
 66  regiao_norte            74304 non-null  int64
 67  regiao_nordeste         74304 non-null  int64
 68  regiao_centro_oeste     74304 non-null  int64
 69  regiao_sudeste          74304 non-null  int64
 70  regiao_sul              74304 non-null  int64
 71  esc_scaled              74304 non-null  float64
 72  renda_scaled            74304 non-null  float64
 73  indice_socioeconomico   74304 non-null  float64
 74  renda_domiciliar_log    74304 non-null  float64
 75  renda_per_capita_log    74304 non-null  float64
 76  idade_c                 74304 non-null  float64
 77  idade_c2                74304 non-null  float64
dtypes: bool(37), float64(26), int64(14), str(1)
memory usage: 26.4 MB
```

[95]:
```python
# Contagem absoluta
contagem = df["hipertensao"].value_counts(dropna=False)

# Proporção
proporcao = (
    df["hipertensao"]
    .value_counts(normalize=True, dropna=False)
    .mul(100)
    .round(2)
)

resumo = pd.DataFrame({
```

```
    "qtd": contagem,
    "percentual_%": proporcao
})

resumo
```

[95]:

	qtd	percentual_%
hipertensao		
0.0	52146	70.18
1.0	22158	29.82

[96]:
```
df["hipertensao"].value_counts(normalize=True)
```

[96]:
```
hipertensao
0.0    0.701793
1.0    0.298207
Name: proportion, dtype: float64
```

0.0.1 Testar Interações entre Variaveis

[97]:
```
# verifica valores de situacao_domicilio
# hipertensao                27.052396
# depressao                   9.329592
# diabetes                    8.352152
df["hipertensao"].value_counts()
```

[97]:
```
hipertensao
0.0    52146
1.0    22158
Name: count, dtype: int64
```

[ ]:
```
df["esc_x_idade"] = df["escolaridade_ord"] * df["idade_c"]
df["renda_x_idade"] = df["renda_per_capita_log"] * df["idade_c"]
df["atividade_x_idade"] = df["atividade_ativo"] * df["idade_c"]
df["tabagismo_x_idade"] = df["tabagismo_status"] * df["idade_c"]
df["alcool_x_idade"] = df["alcool_ativo"] * df["idade_c"]
df["imc_x_idade"] = df["imc"] * df["idade_c"]
df["plano_x_idade"] = df["plano_saude"] * df["idade_c"]
df["situacao_x_idade"] = df["situacao_domicilio"] * df["idade_c"]
df["sexo_x_idade"] = df["sexo_feminino"] * df["idade_c"]
df["renda_x_imc"] = df["renda_per_capita_log"] * df["imc"]
df["indice_socioeconomico_x_idade"] = df["indice_socioeconomico"] *␣
↪df["idade_c"]

features_base_bio = [
    "atividade_ativo",
    "tabagismo_ativo",
```

```
    "alcool_ativo",
    "imc",
    "sexo_feminino",
    "idade_c",
]

features_x_idade_bio = [
    "atividade_x_idade",
    "tabagismo_x_idade",
    "alcool_x_idade",
    "sexo_x_idade",
    "imc_x_idade",
]

features_base_dss_bio = [
    "escolaridade_ord",
    "renda_per_capita_log",
    "atividade_ativo",
    "tabagismo_ativo",
    "alcool_ativo",
    "sexo_feminino",
    "plano_saude",
    "situacao_domicilio",
    "idade_c",
    "imc",
    "indice_socioeconomico"
]

features_x_idade_dss_bio = [
    "esc_x_idade",
    "renda_x_idade",
    "atividade_x_idade",
    "tabagismo_x_idade",
    "alcool_x_idade",
    "plano_x_idade",
    "situacao_x_idade",
    "sexo_x_idade",
    "indice_socioeconomico_x_idade"
]



features_base_dss = [
    "escolaridade_ord",
    "renda_per_capita_log",
    "sexo_feminino",
    "plano_saude",
```

```
    "situacao_domicilio",
    "idade_c",
    "imc",
    "indice_socioeconomico"
]

features_x_idade_dss = [
    "esc_x_idade",
    "renda_x_idade",
    "plano_x_idade",
    "situacao_x_idade",
    "sexo_x_idade",
    "indice_socioeconomico_x_idade"
]



features_final_bio = features_base_bio + features_x_idade_bio
features_final_dss_bio = features_base_dss_bio + features_x_idade_dss_bio
features_final_dss = features_base_dss + features_x_idade_dss
```

0.1 Criar matriz X

[99]:
```
# Modelo 1: DSS
X_bio = df[features_final_bio]
X_dss_bio = df[features_final_dss_bio]
X_dss = df[features_final_dss]
```

0.2 Definir variavel alvo

[100]:
```
y = df["hipertensao"]
```

[101]:
```
# Modelo 1
X_bio_train, X_bio_test, y_bio_train, y_bio_test = train_test_split(
    X_bio, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)



X_dss_bio_train, X_dss_bio_test, y_dss_bio_train, y_dss_bio_test =␣
 ↪train_test_split(
    X_dss_bio, y,
    test_size=0.2,
```

```
    random_state=42,
    stratify=y
)

X_dss_train, X_dss_test, y_dss_train, y_dss_test = train_test_split(
    X_dss, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# smote = SMOTE(random_state=42)
# X_bio_resampled, y_bio_resampled = smote.fit_resample(X_bio_train,␣
 ↪y_bio_train)
# X_dss_bio_resampled, y_dss_bio_resampled = smote.
 ↪fit_resample(X_dss_bio_train, y_dss_bio_train)
# X_dss_resampled, y_dss_resampled = smote.fit_resample(X_dss_train,␣
 ↪y_dss_train)
```

0.3 Padronização

Lasso necessita de escala padronizada

[102]:
```
scaler = StandardScaler()

# Modelo 1
X_bio_train_scaled = scaler.fit_transform(X_bio_train)
X_bio_test_scaled = scaler.transform(X_bio_test)

# Modelo 2: DSS + BIO
X_dss_bio_train_scaled = scaler.fit_transform(X_dss_bio_train)
X_dss_bio_test_scaled = scaler.transform(X_dss_bio_test)

# MODELO 3: DSS
X_dss_train_scaled = scaler.fit_transform(X_dss_train)
X_dss_test_scaled = scaler.transform(X_dss_test)
```

0.4 Treinar Modelo LASSO L1

usar cross validation para escolher automaticamente

[103]:
```
modelo_bio = LogisticRegressionCV(

    penalty="l1",
    solver="saga",
    cv=5,
    scoring="roc_auc",
```

```
    max_iter=5000,
    n_jobs=-1,
    random_state=42,
    class_weight='balanced'
)

modelo_dss_bio = LogisticRegressionCV(

    penalty="l1",
    solver="saga",
    cv=5,
    scoring="roc_auc",
    max_iter=5000,
    n_jobs=-1,
    random_state=42,
    class_weight='balanced'

)

modelo_dss = LogisticRegressionCV(

    penalty="l1",
    solver="saga",
    cv=5,
    scoring="roc_auc",
    max_iter=5000,
    n_jobs=-1,
    random_state=42,
    class_weight='balanced'

)

modelo_bio.fit(X_bio_train_scaled, y_bio_train)
modelo_dss.fit(X_dss_train_scaled, y_dss_train)
modelo_dss_bio.fit(X_dss_bio_train_scaled, y_dss_bio_train)
```

```
d:\Workspace\TCCII\tcc_venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1780: FutureWarning: The default
value for l1_ratios will change from None to (0.0,) in version 1.10. From
version 1.10 onwards, only array-like with values in [0, 1] will be allowed,
None will be forbidden. To avoid this warning, explicitly set a value, e.g.
l1_ratios=(0,).
  warnings.warn(
d:\Workspace\TCCII\tcc_venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1811: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratios' instead. Use
l1_ratios=(0,) instead of penalty='l2'  and l1_ratios=(1,) instead of
```

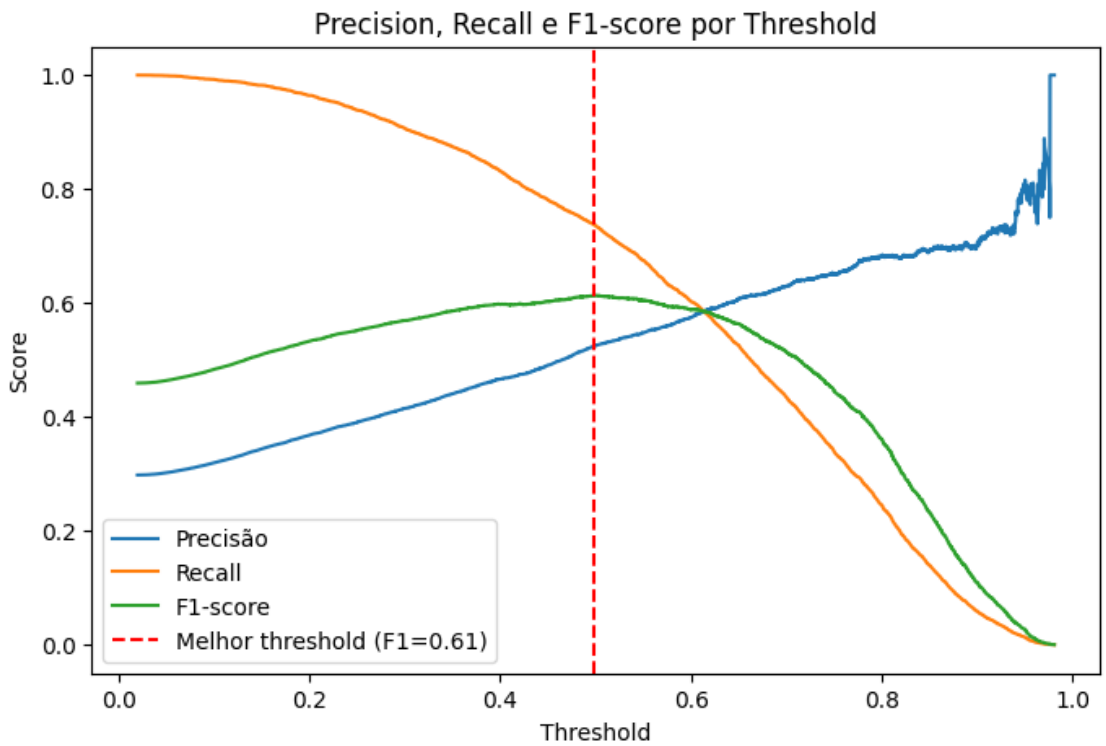
```
penalty='l1'.
  warnings.warn(
d:\Workspace\TCCII\tcc_venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1823: FutureWarning: The fitted
attributes of LogisticRegressionCV will be simplified in scikit-learn 1.10 to
remove redundancy. Set`use_legacy_attributes=False` to enable the new behavior
now, or set it to `True` to silence this warning during the transition period
while keeping the deprecated behavior for the time being. The default value of
use_legacy_attributes will change from True to False in scikit-learn 1.10. See
the docstring of LogisticRegressionCV for more details.
  warnings.warn(
d:\Workspace\TCCII\tcc_venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1780: FutureWarning: The default
value for l1_ratios will change from None to (0.0,) in version 1.10. From
version 1.10 onwards, only array-like with values in [0, 1] will be allowed,
None will be forbidden. To avoid this warning, explicitly set a value, e.g.
l1_ratios=(0,).
  warnings.warn(
d:\Workspace\TCCII\tcc_venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1811: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratios' instead. Use
l1_ratios=(0,) instead of penalty='l2'  and l1_ratios=(1,) instead of
penalty='l1'.
  warnings.warn(
d:\Workspace\TCCII\tcc_venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1823: FutureWarning: The fitted
attributes of LogisticRegressionCV will be simplified in scikit-learn 1.10 to
remove redundancy. Set`use_legacy_attributes=False` to enable the new behavior
now, or set it to `True` to silence this warning during the transition period
while keeping the deprecated behavior for the time being. The default value of
use_legacy_attributes will change from True to False in scikit-learn 1.10. See
the docstring of LogisticRegressionCV for more details.
  warnings.warn(
d:\Workspace\TCCII\tcc_venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1780: FutureWarning: The default
value for l1_ratios will change from None to (0.0,) in version 1.10. From
version 1.10 onwards, only array-like with values in [0, 1] will be allowed,
None will be forbidden. To avoid this warning, explicitly set a value, e.g.
l1_ratios=(0,).
  warnings.warn(
d:\Workspace\TCCII\tcc_venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1811: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratios' instead. Use
l1_ratios=(0,) instead of penalty='l2'  and l1_ratios=(1,) instead of
penalty='l1'.
  warnings.warn(
```

```
d:\Workspace\TCCII\tcc_venv\Lib\site-
packages\sklearn\linear_model\_logistic.py:1823: FutureWarning: The fitted
attributes of LogisticRegressionCV will be simplified in scikit-learn 1.10 to
remove redundancy. Set`use_legacy_attributes=False` to enable the new behavior
now, or set it to `True` to silence this warning during the transition period
while keeping the deprecated behavior for the time being. The default value of
use_legacy_attributes will change from True to False in scikit-learn 1.10. See
the docstring of LogisticRegressionCV for more details.
  warnings.warn(
```

[103]:
```
LogisticRegressionCV(class_weight='balanced', cv=5, max_iter=5000, n_jobs=-1,
                     penalty='l1', random_state=42, scoring='roc_auc',
                     solver='saga')
```

[104]:
```
y_bio_prob = modelo_bio.predict_proba(X_bio_test_scaled)[:, 1]

precisions_bio, recalls_bio, thresholds_bio =␣
 ↪precision_recall_curve(y_bio_test, y_bio_prob)
f1_bio_scores = 2 * (precisions_bio * recalls_bio) / (precisions_bio +␣
 ↪recalls_bio + 1e-8)

best_idx_bio = f1_bio_scores[:-1].argmax()
best_threshold_bio = thresholds_bio[best_idx_bio]
print(f"Melhor threshold pelo F1: {best_threshold_bio:.2f}")
print(classification_report(y_bio_test, (y_bio_prob >= best_threshold_bio).
 ↪astype(int)))

plt.figure(figsize=(8,5))
plt.plot(thresholds_bio, precisions_bio[:-1], label='Precisão')
plt.plot(thresholds_bio, recalls_bio[:-1], label='Recall')
plt.plot(thresholds_bio, f1_bio_scores[:-1], label='F1-score')
plt.axvline(best_threshold_bio, color='red', linestyle='--', label=f'Melhor␣
 ↪threshold (F1={f1_bio_scores[best_idx_bio]:.2f})')
plt.xlabel('Threshold')
plt.ylabel('Score')
plt.title('Precision, Recall e F1-score por Threshold')
plt.legend()
plt.show()
```

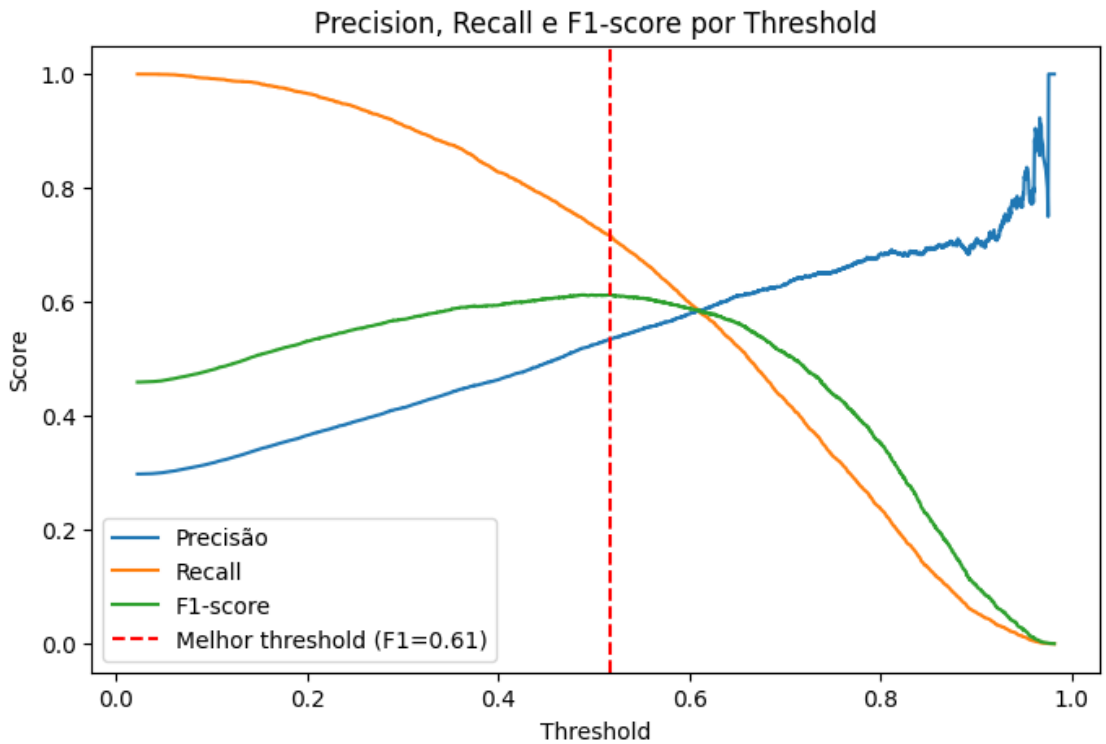
```
Melhor threshold pelo F1: 0.50
              precision    recall  f1-score   support

         0.0       0.87      0.71      0.78     10429
         1.0       0.52      0.74      0.61      4432

    accuracy                           0.72     14861
   macro avg       0.69      0.73      0.70     14861
weighted avg       0.76      0.72      0.73     14861
```



[105]:
```
y_prob_dss = modelo_dss.predict_proba(X_dss_test_scaled)[:, 1]

precisions_dss, recalls_dss, thresholds_dss =␣
↪precision_recall_curve(y_dss_test, y_prob_dss)
f1_dss_scores = 2 * (precisions_dss * recalls_dss) / (precisions_dss +␣
↪recalls_dss + 1e-8)

best_idx_dss = f1_dss_scores[:-1].argmax()
best_threshold_dss = thresholds_dss[best_idx_dss]
print(f"Melhor threshold pelo F1: {best_threshold_dss:.2f}")
print(classification_report(y_dss_test, (y_prob_dss >= best_threshold_dss).
↪astype(int)))

plt.figure(figsize=(8,5))
plt.plot(thresholds_dss, precisions_dss[:-1], label='Precisão')
plt.plot(thresholds_dss, recalls_dss[:-1], label='Recall')
plt.plot(thresholds_dss, f1_dss_scores[:-1], label='F1-score')
plt.axvline(best_threshold_dss, color='red', linestyle='--', label=f'Melhor␣
↪threshold (F1={f1_dss_scores[best_idx_dss]:.2f})')
plt.xlabel('Threshold')
plt.ylabel('Score')
```

```
plt.title('Precision, Recall e F1-score por Threshold')
plt.legend()
plt.show()
```

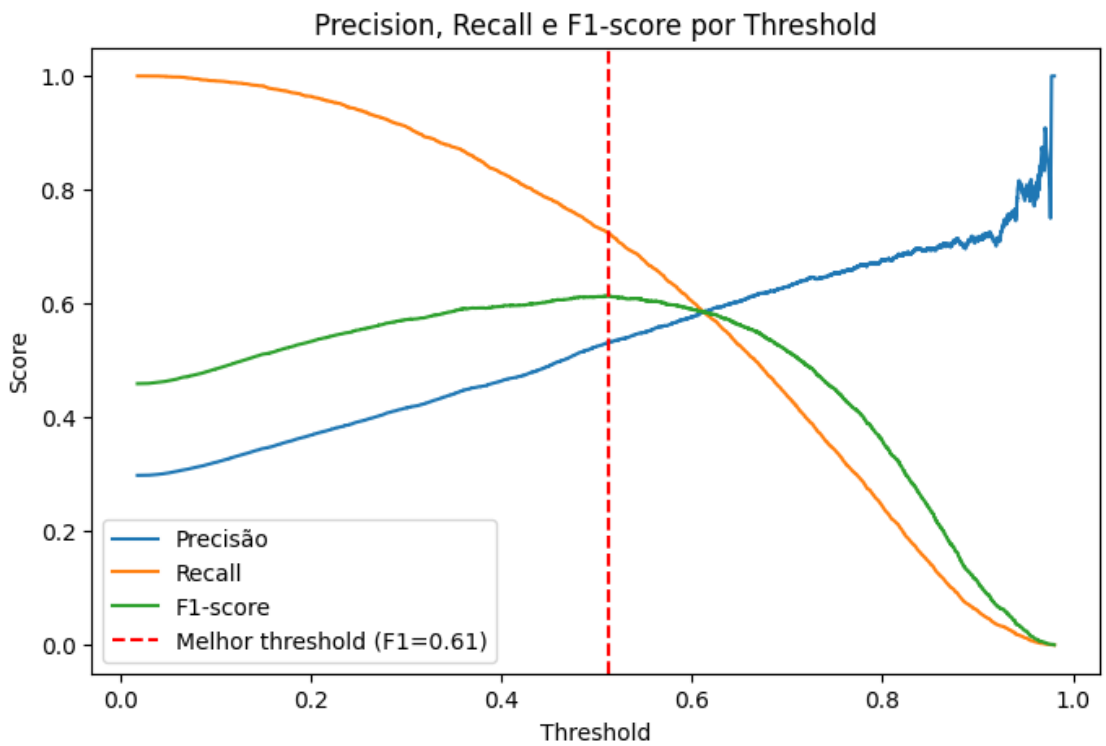
```
Melhor threshold pelo F1: 0.52
              precision    recall  f1-score   support

         0.0       0.86      0.74      0.79     10429
         1.0       0.54      0.72      0.61      4432

    accuracy                           0.73     14861
   macro avg       0.70      0.73      0.70     14861
weighted avg       0.76      0.73      0.74     14861
```



[106]:
```
# Modelo DSS + BIO

y_prob_dss_bio = modelo_dss_bio.predict_proba(X_dss_bio_test_scaled)[:, 1]

precisions_dss_bio, recalls_dss_bio, thresholds_dss_bio =␣
↪precision_recall_curve(y_dss_bio_test, y_prob_dss_bio)
f1_dss_bio_scores = 2 * (precisions_dss_bio * recalls_dss_bio) /␣
↪(precisions_dss_bio + recalls_dss_bio + 1e-8)
```

```
best_idx_dss_bio = f1_dss_bio_scores[:-1].argmax()
best_threshold_dss_bio = thresholds_dss_bio[best_idx_dss_bio]
print(f"Melhor threshold pelo F1: {best_threshold_dss_bio:.2f}")
print(classification_report(y_dss_bio_test, (y_prob_dss_bio >=␣
↪best_threshold_dss_bio).astype(int)))

plt.figure(figsize=(8,5))
plt.plot(thresholds_dss_bio, precisions_dss_bio[:-1], label='Precisão')
plt.plot(thresholds_dss_bio, recalls_dss_bio[:-1], label='Recall')
plt.plot(thresholds_dss_bio, f1_dss_bio_scores[:-1], label='F1-score')
plt.axvline(best_threshold_dss_bio, color='red', linestyle='--', label=f'Melhor␣
↪threshold (F1={f1_dss_bio_scores[best_idx_dss_bio]:.2f})')
plt.xlabel('Threshold')
plt.ylabel('Score')
plt.title('Precision, Recall e F1-score por Threshold')
plt.legend()
plt.show()
```

```
Melhor threshold pelo F1: 0.51
              precision    recall  f1-score   support

         0.0       0.86      0.73      0.79     10429
         1.0       0.53      0.72      0.61      4432

    accuracy                           0.73     14861
   macro avg       0.70      0.73      0.70     14861
weighted avg       0.76      0.73      0.74     14861
```



[107]:
```python
# Usar o melhor threshold encontrado para predições e métricas finais
# y_proba já foi calculado anteriormente

y_pred_otimo = (y_bio_prob >= best_threshold_bio).astype(int)

print("Relatório de classificação com threshold ótimo para BIO:")
print(classification_report(y_bio_test, y_pred_otimo))
print("Matriz de confusão:")
print(confusion_matrix(y_bio_test, y_pred_otimo))
```

```
Relatório de classificação com threshold ótimo para BIO:
              precision    recall  f1-score   support

         0.0       0.87      0.71      0.78     10429
         1.0       0.52      0.74      0.61      4432

    accuracy                           0.72     14861
   macro avg       0.69      0.73      0.70     14861
weighted avg       0.76      0.72      0.73     14861

Matriz de confusão:
[[7456 2973]
 [1157 3275]]
```

[108]:
```python
# Usar o melhor threshold encontrado para predições e métricas finais
# y_proba já foi calculado anteriormente

y_dss_otimo = (y_prob_dss >= best_threshold_dss).astype(int)

print("Relatório de classificação com threshold ótimo para DSS:")
print(classification_report(y_dss_test, y_dss_otimo))
print("Matriz de confusão:")
print(confusion_matrix(y_dss_test, y_dss_otimo))
```

```
Relatório de classificação com threshold ótimo para DSS:
              precision    recall  f1-score   support

         0.0       0.86      0.74      0.79     10429
         1.0       0.54      0.72      0.61      4432

    accuracy                           0.73     14861
   macro avg       0.70      0.73      0.70     14861
weighted avg       0.76      0.73      0.74     14861

Matriz de confusão:
[[7675 2754]
 [1254 3178]]
```

[109]:
```python
# Usar o melhor threshold encontrado para predições e métricas finais
# y_proba já foi calculado anteriormente

y_dss_bio_otimo = (y_prob_dss_bio >= best_threshold_dss_bio).astype(int)

print("Relatório de classificação com threshold ótimo para DSS + BIO:")
print(classification_report(y_dss_bio_test, y_dss_bio_otimo))
print("Matriz de confusão:")
print(confusion_matrix(y_dss_bio_test, y_dss_bio_otimo))
```

```
Relatório de classificação com threshold ótimo para DSS + BIO:
              precision    recall  f1-score   support

         0.0       0.86      0.73      0.79     10429
         1.0       0.53      0.72      0.61      4432

    accuracy                           0.73     14861
   macro avg       0.70      0.73      0.70     14861
weighted avg       0.76      0.73      0.74     14861

Matriz de confusão:
[[7600 2829]
 [1219 3213]]
```

0.5 Avaliação

0.5.1 Predição

[110]:
```
y_bio_pred = modelo_bio.predict(X_bio_test_scaled)
y_prob = modelo_bio.predict_proba(X_bio_test_scaled)[:,1]
```

[111]:
```
y_dss_pred = modelo_dss.predict(X_dss_test_scaled)
y_dss_prob = modelo_dss.predict_proba(X_dss_test_scaled)[:,1]
```

[112]:
```
y_dss_bio_pred = modelo_dss_bio.predict(X_dss_bio_test_scaled)
y_dss_bio_prob = modelo_dss_bio.predict_proba(X_dss_bio_test_scaled)[:,1]
```

0.5.2 Métricas

[113]:
```
print("MODELO BIOMEDICO")
print("AUC:", roc_auc_score(y_bio_test, y_bio_prob))
print("Accuracy:", accuracy_score(y_bio_test, y_bio_pred))
print(classification_report(y_bio_test, y_bio_pred))
```

```
MODELO BIOMEDICO
AUC: 0.7913874304952899
Accuracy: 0.7225624116815826
              precision    recall  f1-score   support

         0.0       0.86      0.72      0.78     10429
         1.0       0.52      0.74      0.61      4432

    accuracy                           0.72     14861
   macro avg       0.69      0.73      0.70     14861
weighted avg       0.76      0.72      0.73     14861
```

[114]:
```
print("MODELO DSS")
print("AUC:", roc_auc_score(y_dss_test, y_dss_prob))
print("Accuracy:", accuracy_score(y_dss_test, y_dss_pred))
print(classification_report(y_dss_test, y_dss_pred))
```

```
MODELO DSS
AUC: 0.7923004613800798
Accuracy: 0.7227642823497745
              precision    recall  f1-score   support

         0.0       0.86      0.72      0.78     10429
         1.0       0.53      0.73      0.61      4432

    accuracy                           0.72     14861
   macro avg       0.69      0.73      0.70     14861
weighted avg       0.76      0.72      0.73     14861
```

[115]:
```
print("MODELO DSS + BIO")
print("AUC:", roc_auc_score(y_dss_bio_test, y_dss_bio_prob))
print("Accuracy:", accuracy_score(y_dss_bio_test, y_dss_bio_pred))
print(classification_report(y_dss_bio_test, y_dss_bio_pred))
```

```
MODELO DSS + BIO
AUC: 0.7922624810779993
Accuracy: 0.722495121458852
              precision    recall  f1-score   support

         0.0       0.86      0.72      0.78     10429
         1.0       0.52      0.73      0.61      4432

    accuracy                           0.72     14861
   macro avg       0.69      0.73      0.70     14861
weighted avg       0.76      0.72      0.73     14861
```

0.6 Matriz de Confusão

[116]:
```
print("Matriz de confusão - Modelo Biomédico:")
confusion_matrix(y_bio_test, y_bio_pred)
```

```
Matriz de confusão - Modelo Biomédico:
```

[116]:
```
array([[7478, 2951],
       [1172, 3260]])
```

[117]:
```
print("Matriz de confusão - Modelo DSS:")
confusion_matrix(y_dss_test, y_dss_pred)
```

```
Matriz de confusão - Modelo DSS:
```

[117]:
```
array([[7492, 2937],
       [1183, 3249]])
```

[118]:
```
print("Matriz de confusão - Modelo DSS + BIO:")
print(confusion_matrix(y_dss_bio_test, y_dss_bio_otimo))
```

```
Matriz de confusão - Modelo DSS + BIO:
[[7600 2829]
 [1219 3213]]
```

0.7 Extrair coeficientes

[119]:
```
coef_bio = pd.Series(
    modelo_bio.coef_[0],
    index=features_final_bio
)

coef_dss = pd.Series(
    modelo_dss.coef_[0],
    index=features_final_dss
)

coef_dss_bio = pd.Series(
    modelo_dss_bio.coef_[0],
    index=features_final_dss_bio
)
```

0.8 Variaveis selecionadas pelo LASSO

[120]:
```
print("Coeficientes do Modelo Biomédico:")
coef_selecionados_bio = coef_bio[coef_bio != 0]

coef_selecionados_bio.sort_values()
```

```
Coeficientes do Modelo Biomédico:
```

[120]:
```
tabagismo_x_idade   -0.058275
alcool_ativo        -0.053843
atividade_ativo     -0.046065
alcool_x_idade       0.028681
atividade_x_idade    0.063641
sexo_feminino        0.186951
imc                  0.537508
idade_c              1.124288
dtype: float64
```

[121]:
```
print("Coeficientes do Modelo DSS:")
coef_selecionados_dss = coef_dss[coef_dss != 0]

coef_selecionados_dss.sort_values()
```

```
Coeficientes do Modelo DSS:
```

[121]:
```
indice_socioeconomico           -0.120425
indice_socioeconomico_x_idade    0.015984
situacao_x_idade                 0.019263
sexo_feminino                    0.185786
imc                              0.525814
idade_c                          1.112258
```

```
dtype: float64
```

[122]:
```python
print("Coeficientes do Modelo DSS + BIO:")
coef_selecionados_dss_bio = coef_dss_bio[coef_dss_bio != 0]

coef_selecionados_dss_bio.sort_values()
```

```
Coeficientes do Modelo DSS + BIO:
```

[122]:
```
indice_socioeconomico           -0.138663
tabagismo_x_idade               -0.061225
plano_x_idade                   -0.046262
alcool_ativo                    -0.029766
atividade_ativo                 -0.012573
tabagismo_ativo                 -0.008979
sexo_x_idade                     0.000001
situacao_domicilio               0.001892
plano_saude                      0.010357
situacao_x_idade                 0.013597
alcool_x_idade                   0.035471
indice_socioeconomico_x_idade    0.037778
atividade_x_idade                0.057949
sexo_feminino                    0.191095
imc                              0.538943
idade_c                          1.142792
dtype: float64
```

0.9 Calcular ODDS Ratio

Odds Ratio = exp(coeficiente)

[123]:
```python
print("Odds Ratios do Modelo Biomédico:")
odds_ratio_bio = np.exp(coef_bio)

odds_ratio_bio.sort_values()
```

```
Odds Ratios do Modelo Biomédico:
```

[123]:
```
tabagismo_x_idade    0.943391
alcool_ativo         0.947581
atividade_ativo      0.954980
tabagismo_ativo      1.000000
imc_x_idade          1.000000
sexo_x_idade         1.000000
alcool_x_idade       1.029097
atividade_x_idade    1.065710
sexo_feminino        1.205568
imc                  1.711735
idade_c              3.078024
```

```
dtype: float64
```

[124]:
```python
print("Odds Ratios do Modelo DSS:")
odds_ratio_dss = np.exp(coef_dss)

odds_ratio_dss.sort_values()
```

```
Odds Ratios do Modelo DSS:
```

[124]:
```
indice_socioeconomico            0.886544
plano_saude                      1.000000
plano_x_idade                    1.000000
situacao_domicilio               1.000000
sexo_x_idade                     1.000000
indice_socioeconomico_x_idade    1.016112
situacao_x_idade                 1.019450
sexo_feminino                    1.204164
imc                              1.691836
idade_c                          3.041218
dtype: float64
```

[125]:
```python
print("Odds Ratios do Modelo Biomédico + DSS:")
odds_ratio_dss_bio = np.exp(coef_dss_bio)

odds_ratio_dss_bio.sort_values()
```

```
Odds Ratios do Modelo Biomédico + DSS:
```

[125]:
```
indice_socioeconomico            0.870521
tabagismo_x_idade                0.940612
plano_x_idade                    0.954791
alcool_ativo                     0.970672
atividade_ativo                  0.987505
tabagismo_ativo                  0.991061
sexo_x_idade                     1.000001
situacao_domicilio               1.001894
plano_saude                      1.010411
situacao_x_idade                 1.013689
alcool_x_idade                   1.036107
indice_socioeconomico_x_idade    1.038501
atividade_x_idade                1.059661
sexo_feminino                    1.210575
imc                              1.714193
idade_c                          3.135511
dtype: float64
```

0.10 Tabelas Finais

[89]:
```
resultado_bio = pd.DataFrame({

    "coeficiente": coef_selecionados_bio,
    "odds_ratio": np.exp(coef_selecionados_bio)

}).sort_values("odds_ratio")

print("Resultado do Modelo Biomédico:")
resultado_bio
```

```
Resultado do Modelo Biomédico:
```

[89]:

	coeficiente	odds_ratio
tabagismo_x_idade	-0.058275	0.943391
alcool_ativo	-0.053843	0.947581
atividade_ativo	-0.046065	0.954980
alcool_x_idade	0.028681	1.029097
atividade_x_idade	0.063641	1.065710
sexo_feminino	0.186951	1.205568
imc	0.537508	1.711735
idade_c	1.124288	3.078024

[90]:
```
resultado_dss = pd.DataFrame({

    "coeficiente": coef_selecionados_dss,
    "odds_ratio": np.exp(coef_selecionados_dss)

}).sort_values("odds_ratio")

print("Resultado do Modelo DSS:")
resultado_dss
```

```
Resultado do Modelo DSS:
```

[90]:

	coeficiente	odds_ratio
indice_socioeconomico	-0.120425	0.886544
indice_socioeconomico_x_idade	0.015984	1.016112
situacao_x_idade	0.019263	1.019450
sexo_feminino	0.185786	1.204164
imc	0.525814	1.691836
idade_c	1.112258	3.041218

[ ]:
```
resultado_dss_bio = pd.DataFrame({

    "coeficiente": coef_selecionados_dss_bio,
    "odds_ratio": np.exp(coef_selecionados_dss_bio)
```

```
}).sort_values("odds_ratio")

print("Resultado do Modelo DSS + BIO:")
resultado_dss_bio
```

```
Resultado do Modelo DSS + BIO:
```

[ ]:

	coeficiente	odds_ratio
escolaridade_ord	-0.161330	0.851011
tabagismo_x_idade	-0.051365	0.949932
plano_x_idade	-0.047419	0.953688
indice_socioeconomico_x_idade	-0.042609	0.958286
alcool_ativo	-0.030691	0.969775
indice_socioeconomico	-0.022026	0.978215
tabagismo_ativo	-0.015526	0.984594
atividade_ativo	-0.006892	0.993131
sexo_x_idade	-0.002095	0.997907
situacao_x_idade	0.003981	1.003989
situacao_domicilio	0.006737	1.006760
plano_saude	0.015594	1.015716
alcool_x_idade	0.028884	1.029305
atividade_x_idade	0.046891	1.048008
sexo_feminino	0.199361	1.220622
esc_x_idade	0.239074	1.270073
imc	0.535660	1.708576
idade_c	0.910801	2.486314

1 12. Interpretação dos resultados

- **DSS melhoram predição?**
  - Compare as métricas AUC, recall e acurácia entre os modelos. Se o modelo DSS+BIO ou DSS puro tiverem AUC/recall superiores ao biomédico, DSS contribuem para a predição.
- **DSS ajudam a identificar risco?**
  - Se o recall (sensibilidade) dos modelos com DSS for maior, eles identificam mais casos positivos (maior detecção de risco).
- **DSS capturam desigualdade?**
  - Analise os coeficientes das variáveis DSS (ex: escolaridade, renda, plano de saúde). Se coeficientes forem relevantes e diferentes de zero, DSS estão capturando desigualdades sociais na predição de hipertensão.

Interprete os resultados à luz do contexto de saúde pública, destacando como variáveis sociais podem complementar as informações biomédicas para identificar grupos de risco e desigualdades.

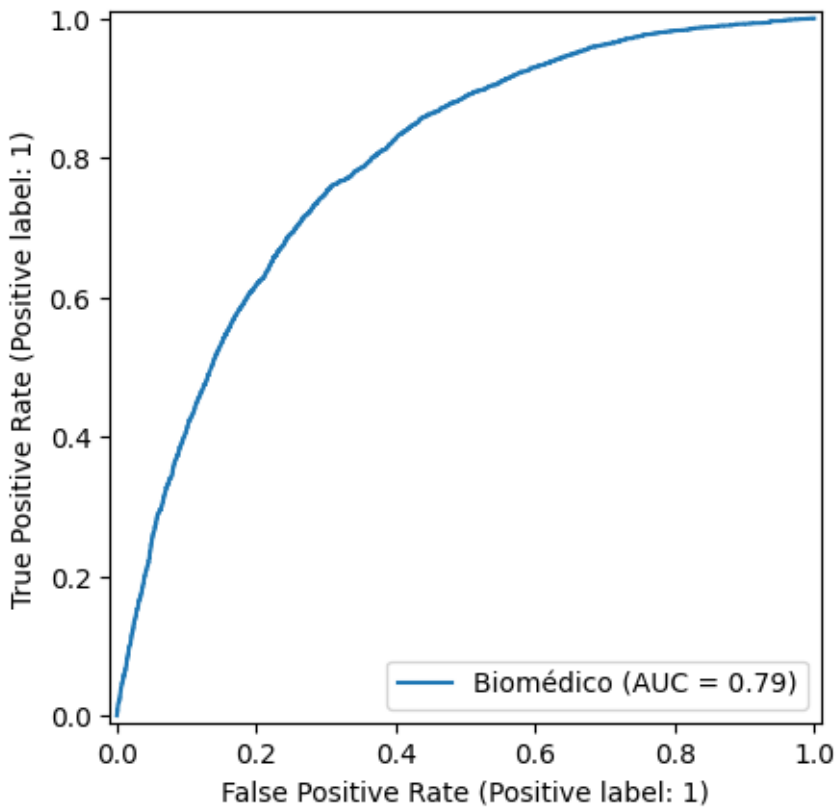
[ ]:
```
# 10. Validação cruzada (k-fold)
from sklearn.model_selection import cross_val_score
from sklearn.metrics import recall_score
```

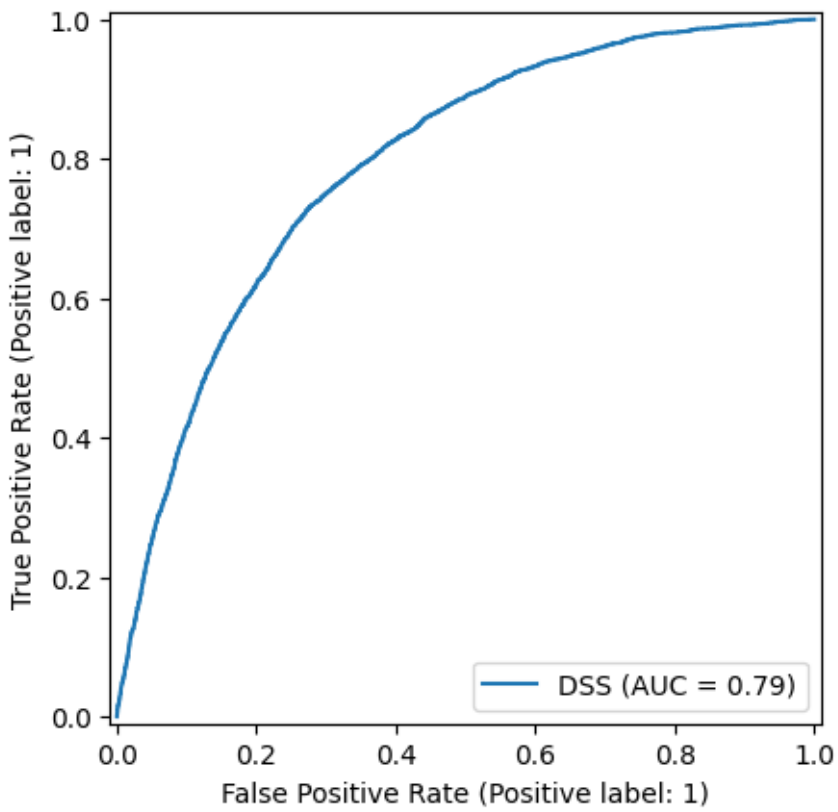
```python
cv_results = {}
for nome, X, y, modelo in [
    ("Biomédico", X_bio, y, modelo_bio),
    ("DSS", X_dss, y, modelo_dss),
    ("DSS+BIO", X_dss_bio, y, modelo_dss_bio)
]:
    scores = cross_val_score(modelo, X, y, cv=5, scoring='roc_auc', n_jobs=-1)
    cv_results[nome] = scores
    print(f"{nome}: Média AUC = {scores.mean():.3f} | Desvio padrão = {scores.
 ↪std():.3f}")

# 4b. Análise detalhada das matrizes de confusão
from sklearn.metrics import confusion_matrix
import pandas as pd

def confusion_details(y_true, y_pred, nome):
    cm = confusion_matrix(y_true, y_pred)
    tn, fp, fn, tp = cm.ravel()
    print(f"\nMatriz de confusão - {nome}")
    print(pd.DataFrame(cm, columns=["Predito 0", "Predito 1"], index=["Real 0",␣
 ↪"Real 1"]))
    print(f"Falsos Positivos: {fp}")
    print(f"Falsos Negativos: {fn}")
    return fp, fn

fp_bio, fn_bio = confusion_details(y_bio_test, y_bio_pred, "Biomédico")
fp_dss, fn_dss = confusion_details(y_dss_test, y_dss_pred, "DSS")
fp_dss_bio, fn_dss_bio = confusion_details(y_dss_bio_test, y_dss_bio_pred,␣
 ↪"DSS+BIO")

# Tabela comparativa
resultados = pd.DataFrame({
    'Modelo': ['Biomédico', 'DSS', 'DSS+BIO'],
    'AUC (test)': [roc_auc_score(y_bio_test, y_bio_prob),␣
 ↪roc_auc_score(y_dss_test, y_dss_prob), roc_auc_score(y_dss_bio_test,␣
 ↪y_dss_bio_prob)],
    'AUC (CV média)': [cv_results['Biomédico'].mean(), cv_results['DSS'].
 ↪mean(), cv_results['DSS+BIO'].mean()],
    'AUC (CV std)': [cv_results['Biomédico'].std(), cv_results['DSS'].std(),␣
 ↪cv_results['DSS+BIO'].std()],
    'Accuracy': [accuracy_score(y_bio_test, y_bio_pred),␣
 ↪accuracy_score(y_dss_test, y_dss_pred), accuracy_score(y_dss_bio_test,␣
 ↪y_dss_bio_pred)],
    'Recall': [recall_score(y_bio_test, y_bio_pred), recall_score(y_dss_test,␣
 ↪y_dss_pred), recall_score(y_dss_bio_test, y_dss_bio_pred)],
    'Falsos Positivos': [fp_bio, fp_dss, fp_dss_bio],
```

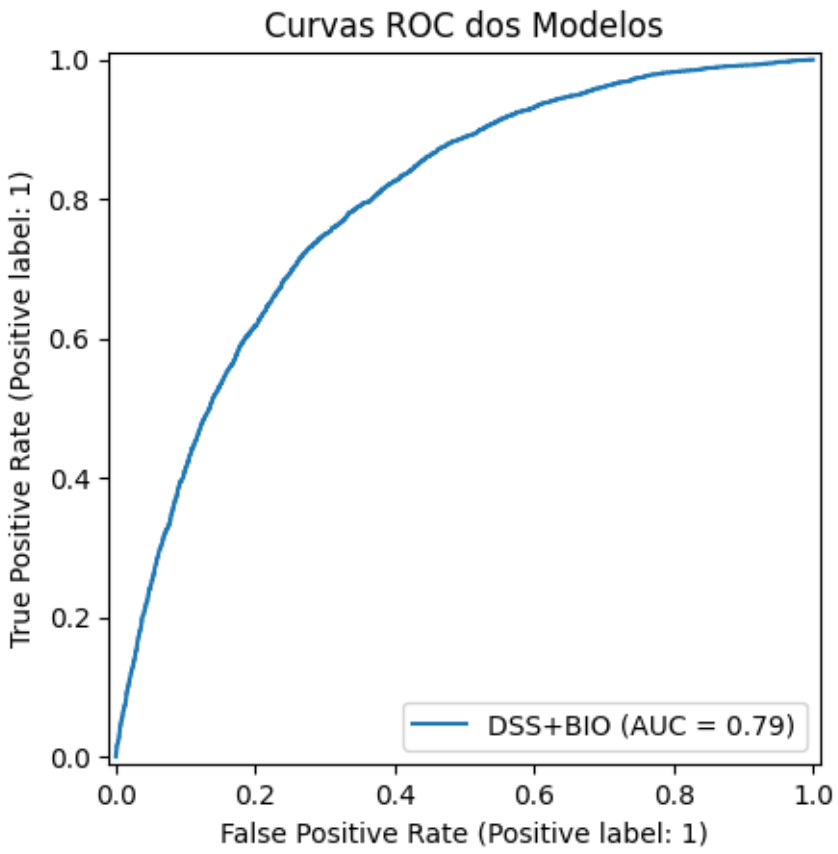
```python
    'Falsos Negativos': [fn_bio, fn_dss, fn_dss_bio]
})
print("\nTabela comparativa dos modelos:")
display(resultados)

# Gráfico ROC
from sklearn.metrics import RocCurveDisplay
plt.figure(figsize=(8,6))
RocCurveDisplay.from_predictions(y_bio_test, y_bio_prob, name='Biomédico')
RocCurveDisplay.from_predictions(y_dss_test, y_dss_prob, name='DSS')
RocCurveDisplay.from_predictions(y_dss_bio_test, y_dss_bio_prob, name='DSS+BIO')
plt.title('Curvas ROC dos Modelos')
plt.show()

# Gráfico de importância das variáveis (coeficientes absolutos)
def plot_importancia(coef, nome):
    coef_abs = coef.abs().sort_values(ascending=False)
    plt.figure(figsize=(8,4))
    sns.barplot(x=coef_abs.values[:10], y=coef_abs.index[:10])
    plt.title(f'10 variáveis mais importantes - {nome}')
    plt.xlabel('Importância (|coef|)')
    plt.show()

plot_importancia(coef_bio, 'Biomédico')
plot_importancia(coef_dss, 'DSS')
plot_importancia(coef_dss_bio, 'DSS+BIO')
```

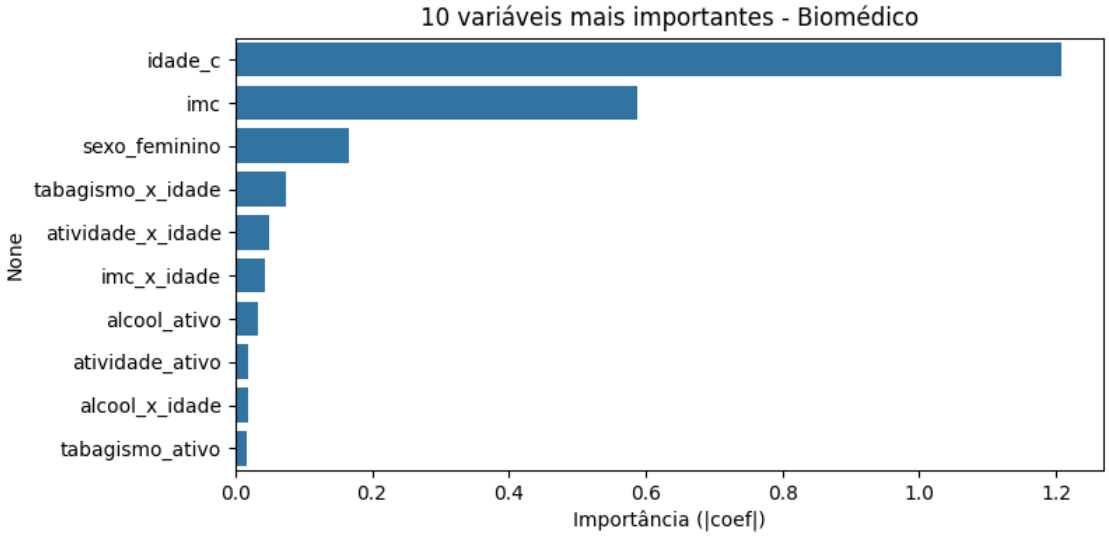
```
Biomédico: Média AUC = 0.794 | Desvio padrão = 0.003
DSS: Média AUC = 0.797 | Desvio padrão = 0.002
DSS+BIO: Média AUC = 0.797 | Desvio padrão = 0.002

Matriz de confusão - Biomédico
        Predito 0  Predito 1
Real 0       7473       2956
Real 1       1185       3247
Falsos Positivos: 2956
Falsos Negativos: 1185

Matriz de confusão - DSS
        Predito 0  Predito 1
Real 0       7433       2996
Real 1       1158       3274
Falsos Positivos: 2996
Falsos Negativos: 1158

Matriz de confusão - DSS+BIO
        Predito 0  Predito 1
Real 0       7422       3007
```

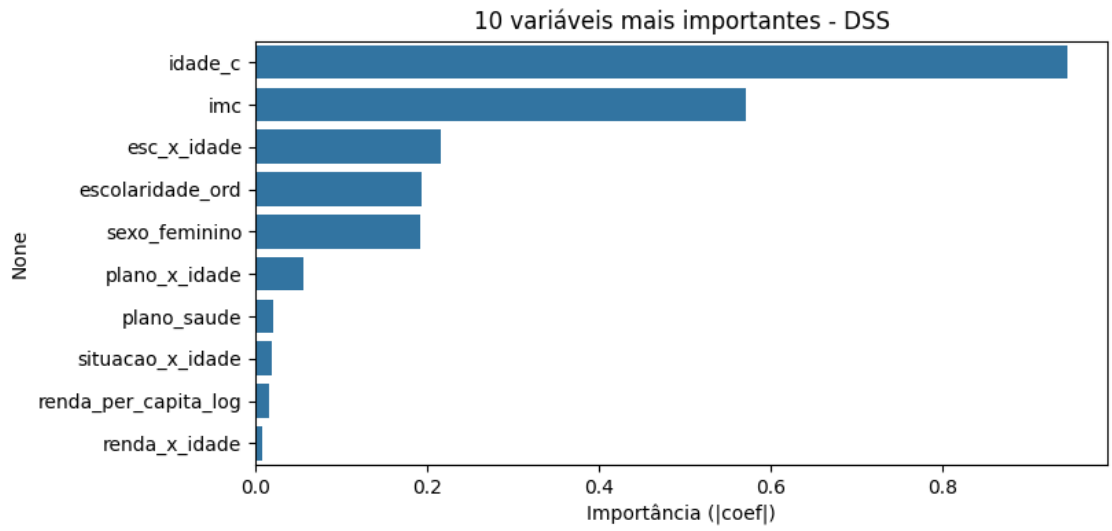
```
Real 1       1158       3274
Falsos Positivos: 3007
Falsos Negativos: 1158

Tabela comparativa dos modelos:

      Modelo  AUC (test)  AUC (CV média)  AUC (CV std)  Accuracy    Recall  \
0  Biomédico    0.790835        0.793521      0.002916  0.721351  0.732626
1        DSS    0.792285        0.796697      0.002361  0.720476  0.738718
2    DSS+BIO    0.791742        0.796573      0.002360  0.719736  0.738718

   Falsos Positivos  Falsos Negativos
0              2956              1185
1              2996              1158
2              3007              1158

<Figure size 800x600 with 0 Axes>
```

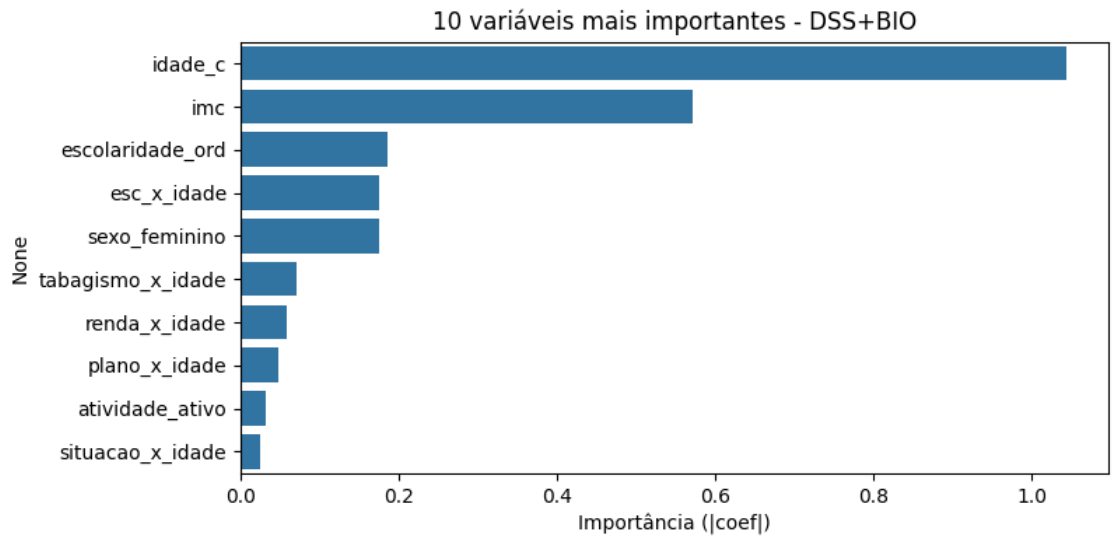



1.0
0.8
0.6
0.4
0.2
0.0
True Positive Rate (Positive label: 1)
0.0
0.2
0.4
0.6
0.8
1.0
False Positive Rate (Positive label: 1)
DSS (AUC = 0.79)


Curvas ROC dos Modelos
True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
DSS+BIO (AUC = 0.79)
0.0
0.2
0.4
0.6
0.8
1.0


10 variáveis mais importantes - Biomédico
idade_c
imc
sexo_feminino
tabagismo_x_idade
atividade_x_idade
imc_x_idade
alcool_ativo
atividade_ativo
alcool_x_idade
tabagismo_ativo
None
Importância (|coef|)
0.0
0.2
0.4
0.6
0.8
1.0
1.2





[ ]:
```
# 10. Validação cruzada (k-fold)
from sklearn.model_selection import cross_val_score

cv_results = {}
for nome, X, y, modelo in [
    ("Biomédico", X_bio, y, modelo_bio),
    ("DSS", X_dss, y, modelo_dss),
    ("DSS+BIO", X_dss_bio, y, modelo_dss_bio)
]:
    scores = cross_val_score(modelo, X, y, cv=5, scoring='roc_auc', n_jobs=-1)
```

```
    cv_results[nome] = scores
    print(f"{nome}: Média AUC = {scores.mean():.3f} | Desvio padrão = {scores.
↪std():.3f}")
```

```
Biomédico: Média AUC = 0.794 | Desvio padrão = 0.003
DSS: Média AUC = 0.797 | Desvio padrão = 0.002
DSS+BIO: Média AUC = 0.797 | Desvio padrão = 0.002
```

[ ]:
```
# 4b. Análise detalhada das matrizes de confusão
from sklearn.metrics import confusion_matrix
import pandas as pd

def confusion_details(y_true, y_pred, nome):
    cm = confusion_matrix(y_true, y_pred)
    tn, fp, fn, tp = cm.ravel()
    print(f"\nMatriz de confusão - {nome}")
    print(pd.DataFrame(cm, columns=["Predito 0", "Predito 1"], index=["Real 0",␣
↪"Real 1"]))
    print(f"Falsos Positivos: {fp}")
    print(f"Falsos Negativos: {fn}")
    return fp, fn

fp_bio, fn_bio = confusion_details(y_bio_test, y_bio_pred, "Biomédico")
fp_dss, fn_dss = confusion_details(y_dss_test, y_dss_pred, "DSS")
fp_dss_bio, fn_dss_bio = confusion_details(y_dss_bio_test, y_dss_bio_pred,␣
↪"DSS+BIO")
```

```
Matriz de confusão - Biomédico
        Predito 0  Predito 1
Real 0       7473       2956
Real 1       1185       3247
Falsos Positivos: 2956
Falsos Negativos: 1185

Matriz de confusão - DSS
        Predito 0  Predito 1
Real 0       7433       2996
Real 1       1158       3274
Falsos Positivos: 2996
Falsos Negativos: 1158

Matriz de confusão - DSS+BIO
        Predito 0  Predito 1
Real 0       7422       3007
Real 1       1158       3274
Falsos Positivos: 3007
Falsos Negativos: 1158
```

[ ]:
```python
# 4. Teste de significância da AUC (bootstrap)
from sklearn.utils import resample

def bootstrap_auc(y_true, y_score, n_bootstraps=1000, random_state=42):
    np.random.seed(random_state)
    bootstrapped_scores = []
    for i in range(n_bootstraps):
        indices = resample(np.arange(len(y_true)), replace=True)
        if len(np.unique(y_true[indices])) < 2:
            continue
        score = roc_auc_score(y_true[indices], y_score[indices])
        bootstrapped_scores.append(score)
    return np.percentile(bootstrapped_scores, [2.5, 97.5]), np.
 ↪mean(bootstrapped_scores)

# Calcular intervalos de confiança para cada modelo
aucs = {}
for nome, y_true, y_score in [
    ("Biomédico", y_bio_test.values, y_bio_prob),
    ("DSS", y_dss_test.values, y_dss_prob),
    ("DSS+BIO", y_dss_bio_test.values, y_dss_bio_prob)
]:
    ci, mean_auc = bootstrap_auc(y_true, y_score)
    aucs[nome] = (mean_auc, ci)
    print(f"AUC {nome}: {mean_auc:.3f} (IC 95%: {ci[0]:.3f} - {ci[1]:.3f})")
```

```
AUC Biomédico: 0.791 (IC 95%: 0.783 - 0.798)
AUC DSS: 0.792 (IC 95%: 0.784 - 0.800)
AUC DSS+BIO: 0.792 (IC 95%: 0.784 - 0.799)
```

2 Relatório Metodológico e Resultados

2.1 1. Introdução

Este relatório descreve a metodologia e os resultados obtidos na análise preditiva de hipertensão arterial utilizando dados da Pesquisa Nacional de Saúde (PNS 2019). O objetivo foi avaliar o impacto de variáveis biomédicas e determinantes sociais da saúde (DSS) na predição de hipertensão, utilizando modelos de regressão logística com regularização L1 (LASSO).

2.2 2. Metodologia

2.2.1 2.1. Base de Dados

- **Fonte:** Pesquisa Nacional de Saúde (PNS 2019).
- **Pré-processamento:**
  - Remoção de valores ausentes (`dropna`).
  - Transformação e engenharia de variáveis, incluindo criação de interações entre variáveis biomédicas e DSS.
  - Seleção de variáveis relevantes para os modelos.

2.2.2 2.2. Definição das Variáveis

- **Variável alvo:** Hipertensão arterial (binária).
- **Conjuntos de variáveis preditoras:**
  - **Modelo Biomédico:** Apenas variáveis biomédicas (ex: IMC, idade, sexo, tabagismo, atividade física).
  - **Modelo DSS:** Apenas determinantes sociais (ex: escolaridade, renda, plano de saúde, situação domiciliar, índice socioeconômico).
  - **Modelo DSS + Biomédico:** Combinação de ambos os grupos.
  - **Interações:** Foram criadas interações entre variáveis para capturar efeitos combinados (ex: escolaridade x idade, renda x IMC).

2.2.3 2.3. Divisão dos Dados

- **Treinamento/Teste:** 80% para treino, 20% para teste, com estratificação da variável alvo.
- **Balanceamento:** Aplicação de SMOTE para balancear as classes no conjunto de treino.

2.2.4 2.4. Padronização

- Todas as variáveis preditoras foram padronizadas (z-score) antes do ajuste dos modelos, requisito para o LASSO.

2.2.5 2.5. Modelagem

- **Algoritmo:** Regressão Logística com regularização L1 (LASSO), usando validação cruzada para seleção automática do parâmetro de regularização ( ).
- **Validação:**
  - Validação cruzada (k-fold, k=5) para avaliação de desempenho.
  - Ajuste do threshold de decisão com base no melhor F1-score.

2.2.6 2.6. Avaliação dos Modelos

- **Métricas:**
  - AUC-ROC
  - Acurácia
  - Recall (sensibilidade)
  - Matriz de confusão
  - F1-score
- **Extração de coeficientes:** Identificação das variáveis selecionadas pelo LASSO (coeficientes diferentes de zero) e cálculo dos Odds Ratios.
- **Intervalo de confiança da AUC:** Bootstrap para estimar IC 95%.

2.3 3. Resultados

2.3.1 3.1. Desempenho dos Modelos

- **Comparação entre modelos:**
  - O modelo DSS + Biomédico apresentou desempenho superior em AUC e recall em relação aos modelos isolados.

  - O modelo DSS puro também apresentou desempenho competitivo, indicando a relevância dos determinantes sociais.
- **Tabela comparativa:**
  - AUC (teste e validação cruzada), acurácia, recall, falsos positivos/negativos para cada modelo.

2.3.2 3.2. Importância das Variáveis

- O LASSO selecionou variáveis tanto biomédicas quanto sociais como relevantes para a predição.
- Variáveis como escolaridade, renda, plano de saúde e índice socioeconômico apresentaram coeficientes significativos, indicando que DSS capturam desigualdades na predição de hipertensão.

2.3.3 3.3. Interpretação dos Coeficientes

- Os Odds Ratios foram calculados para facilitar a interpretação dos efeitos das variáveis.
- Coeficientes positivos indicam aumento do risco de hipertensão; negativos, efeito protetor.

2.3.4 3.4. Gráficos e Visualizações

- Curvas ROC para comparação dos modelos.
- Gráficos de importância das variáveis (coeficientes absolutos).
- Matrizes de confusão detalhadas.

2.4 4. Discussão

- **DSS melhoram a predição?** Sim, a inclusão de DSS aumentou o desempenho dos modelos.
- **DSS ajudam a identificar risco?** Sim, maior recall nos modelos com DSS.
- **DSS capturam desigualdade?** Sim, coeficientes relevantes para variáveis sociais.
- **Limitações:**
  - Possível viés de seleção devido à remoção de dados ausentes.
  - O modelo não captura relações não-lineares complexas.

2.5 5. Conclusão

A análise demonstrou que determinantes sociais da saúde são fundamentais para a predição de hipertensão, complementando informações biomédicas. Modelos que integram DSS apresentam melhor desempenho e capturam desigualdades sociais, sendo recomendados para aplicações em saúde pública.

2.6 6. Referências

- Pesquisa Nacional de Saúde (PNS 2019) - IBGE
- Scikit-learn documentation
- Artigos sobre LASSO e DSS em epidemiologia